

Introdução a Redes Neurais e Redes Convolucionais

Trabalho Individual — Tarefas 1, 2 e 3

Aluno: Lucas Medeiros

Disciplina: Aprendizado Profundo — PPgTI/IMD/UFRN

Docente: Prof. Josenalde Oliveira

Setembro de 2026

Sumário

Tarefa 1 — Estudo Dirigido: TensorFlow Playground	3
Tarefa 2 — Otimização de Hiperparâmetros do MNIST (Optuna)	7
Tarefa 3 — CNNs (PyTorch) para o Multiprova Corretor (EMNIST)	12
Links dos Notebooks e Repositório	15

Tarefa 1 — Estudo Dirigido: TensorFlow Playground

Fiz todos os experimentos abaixo mexendo de verdade no [TensorFlow Playground](#): fui trocando os hiperparâmetros, treinando, resetando e anotando os valores reais de época, perda de treino/teste e como ficava a fronteira de decisão em cada caso.

a) Padrões aprendidos por uma NN (configuração padrão)

Configuração usada: dataset *Circle*, 2 features (X1, X2), 2 camadas ocultas (4 e 2 neurônios), ativação tanh, taxa de aprendizagem 0.03, sem regularização, ruído 0, batch size 10.

A rede converge bem rápido: em 699 épocas cheguei a uma perda de teste de 0.002 e perda de treino de 0.001, com uma fronteira circular bem suave, condizente com a distribuição dos dados. Dá pra ver na prática o que a teoria descreve: os neurônios da primeira camada oculta aprendem padrões mais simples, basicamente retas ou curvas que separam pedaços do espaço, e a segunda camada vai combinando essas saídas até formar a fronteira circular final. Quanto mais camadas e neurônios a rede tem, mais complexos são os padrões que ela consegue compor.

b) Função de ativação

Mantendo a arquitetura de (a), troquei apenas a ativação:

Ativação	Épocas	Perda teste	Perda treino	Fronteira de decisão
Tanh (baseline)	699	0.002	0.001	curva suave (circular)
ReLU	621	0.001	0.000	poligonal/angular, com "quinas"
Linear	1918 (sem melhora)	0.514	0.492	nenhuma fronteira útil

Com ReLU a rede convergiu um pouco mais rápido e a perda final ficou levemente menor, só que a fronteira perdeu a suavidade: como o ReLU é linear por partes, a região de decisão vira um polígono, com quinas nas bordas, enquanto a tanh continua gerando curvas por ser uma função suave.

Já com ativação Linear o resultado foi bem ruim: mesmo depois de quase 2000 épocas o erro não sai de perto de 0.5, o que na prática é chute aleatório. Faz sentido, porque empilhar camadas totalmente lineares dá no mesmo que ter uma única camada linear, não importa quantas camadas eu coloque, e um classificador linear simplesmente não separa um padrão circular, que não é linearmente separável.

c) Taxa de aprendizagem (learning rate)

Mantendo tanh e demais parâmetros fixos:

Learning rate	Épocas	Perda teste	Perda treino	Observação
0.03 (baseline)	699	0.002	0.001	convergência estável
0.00001 (muito baixa)	841	0.512	0.493	quase não aprende
10 (muito alta)	790	0.768	0.816	diverge/instável

Deu pra sentir os dois extremos: uma taxa de aprendizado baixa demais deixa o treino tão devagar que praticamente trava (a perda quase não sai do valor inicial), e uma taxa alta demais faz o gradiente descendente pular o mínimo e divergir. Nesse segundo caso o treino fica instável e a perda pode até aumentar em vez de cair.

d) Risco de mínimos locais (1 camada oculta, 3 neurônios)

Arquitetura reduzida para 1 camada oculta com 3 neurônios (tanh, lr 0.03). Treinei a rede 4 vezes, resetando os pesos entre cada tentativa:

Tentativa	Épocas até parar	Perda teste final	Resultado
1	1019	0.011	convergiu bem
2	1121	0.009	convergiu bem
3	1209	0.009	convergiu bem
4	1152	0.268	preso em mínimo local

Na tentativa 4 a curva de perda para num platô alto e a rede nunca sai dali. Dá pra ver na prática o risco de mínimo local: com poucos neurônios e poucos pesos, a superfície de erro tem bem menos rotas de escape, então dependendo de como os pesos começam, o treino pode simplesmente ficar preso.

e) Poucos neurônios (2 neurônios, 1 camada)

Reduzi ainda mais, para 2 neurônios numa única camada oculta, e testei duas vezes de forma independente. Nas duas, a rede travou exatamente na mesma perda de teste (0.231), com a mesma fronteira em forma de faixa diagonal, mesmo depois de mais de 1200 épocas em ambos os casos. Aqui não é azar de inicialização como no item anterior: a arquitetura simplesmente não tem capacidade suficiente para representar a fronteira circular, então ela sempre converge para a melhor aproximação linear possível.

f) Rede maior (8 neurônios, 1 camada)

Com 8 neurônios numa única camada oculta, repeti o treino duas vezes e as duas convergiram rápido (651 e 659 épocas), com perda de teste em torno de 0.008-0.009, sem nenhum sinal de travar no meio do caminho. Dá pra perceber que redes com mais capacidade tendem a não cair em mínimos locais ruins com tanta facilidade quanto as menores.

g) Dataset espiral, 4 camadas ocultas × 8 neurônios

Troquei para o dataset Spiral, com 4 camadas ocultas de 8 neurônios cada (tanh), e testei três configurações:

- Baseline (lr = 0.03, sem regularização): a perda caiu devagar no começo, ficou num platô por um tempo e só depois despencou, terminando em 0.003 aos 812 épocas.
- Learning rate alta (0.1) com regularização L2 (0.01): a perda ficou travada em 0.480 entre as épocas 319 e 858, com boa parte dos pesos empurrados quase a zero.
- Learning rate alta (0.1) com regularização L1 (0.01): resultado ainda pior, a perda estagnou em 0.501 (ou seja, chute aleatório) já aos 371 épocas, e praticamente todos os pesos zeraram por causa do efeito de esparsificação da L1.

O que fica claro aqui é que, no dataset espiral, misturar learning rate alta com regularização forte é uma combinação ruim: a regularização acaba penalizando os pesos justamente no momento em que a rede mais precisa deles para conseguir modelar a espiral.

Exercícios de fixação

1) Três funções de ativação populares:

- **Sigmoide:** $\sigma(z) = 1 / (1 + e^{-z})$, saída entre 0 e 1.
- **Tanh:** $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$, saída entre -1 e 1, centrada em zero.
- **ReLU:** $\text{ReLU}(z) = \max(0, z)$.

2) MLP com 10 neurônios de entrada, 1 camada oculta com 50 neurônios, saída com 3 neurônios, tudo ReLU (notação de Géron, *Mãos à Obra*, 2ª ed., p. 219–220; m = nº de instâncias no lote):

- a) X: formato (m, 10).
- b) Camada oculta: **Wh** formato (10, 50), **bh** formato (50,).
- c) Camada de saída: **Wo** formato (50, 3), **bo** formato (3,).
- d) Y: formato (m, 3).
- e) $Y = \text{ReLU}(\text{ReLU}(X \cdot W_h + b_h) \cdot W_o + b_o)$

3) Neurônios/ativação na camada de saída:

- **Spam ou não spam:** 1 neurônio, ativação sigmoide.
- **MNIST:** 10 neurônios, ativação softmax.
- **Cotação do dólar amanhã (regressão):** 1 neurônio, sem ativação (linear).

4) Hiperparâmetros e overfitting:

Numa MLP básica dá pra ajustar: número de camadas ocultas, neurônios por camada, função de ativação, learning rate, otimizador, número de épocas, batch size, regularização (L1/L2 ou dropout) e a inicialização dos pesos.

Se o modelo sobreajustar (erro de treino baixo mas erro de validação/teste alto), as saídas mais comuns são: reduzir a complexidade do modelo, adicionar regularização L1/L2 ou dropout, usar early stopping, conseguir mais dados de treino (ou fazer data augmentation), reduzir o número de épocas, normalizar melhor os dados de entrada, ou usar validação cruzada para escolher os hiperparâmetros de forma mais robusta.

Tarefa 2 — Otimização de Hiperparâmetros do MNIST (Optuna)

Notebook: `tarefa2_mnist_optuna.ipynb`, executado localmente em Python 3.13 / Windows, com TensorFlow 2.21.0, em CPU. Usei como base o notebook do professor `mnist_keras.ipynb`.

1. Metodologia

Usei a biblioteca Optuna (algoritmo TPE, maximizando a métrica) para buscar a melhor combinação de hiperparâmetros de uma MLP para classificar o MNIST, otimizando a acurácia de validação (separei 10% do treino pra isso). O espaço de busca ficou assim: número de camadas ocultas (1 a 3), número de neurônios por camada (32, 64 ou 128), função de ativação (relu ou tanh), taxa de dropout (0 a 0.5), taxa de aprendizado do Adam (entre 1e-4 e 1e-2, em escala log) e tamanho do lote (32, 64 ou 128). Cada trial treinou por 8 épocas, e rodei 20 trials no total — um número de épocas baixo de propósito, só pra tornar a busca viável em tempo razoável rodando em CPU.

2. Resultados da busca (20 trials)

Trial	Acc. val.	batch	n_layers	units	ativ.	dropout	lr
0	0.9692	128	1	64	relu	0.499	0.00206
1	0.9650	64	1	32	tanh	0.029	0.00478
2	0.9715	128	3	32	relu	0.017	0.00565
3	0.9645	32	1	64	tanh	0.389	0.00394
4	0.9648	64	2	64	relu	0.458	0.00046
5	0.9228	64	3	32	relu	0.492	0.00062
6	0.9798	32	1	128	relu	0.254	0.00052
7	0.9808	128	2	128	relu	0.1125	0.00243
8	0.9775	32	1	128	tanh	0.230	0.00318
9	0.9753	128	1	64	relu	0.295	0.00313
10	0.9615	128	2	128	tanh	0.132	0.00018
11	0.9805	32	2	128	relu	0.188	0.00096
12	0.9802	32	2	128	relu	0.125	0.00135
13	0.9630	32	2	128	relu	0.143	0.00972
14	0.9782	128	2	128	relu	0.187	0.00117
15	0.9782	32	3	128	relu	0.093	0.00019
16	0.9803	128	2	128	relu	0.333	0.00107
17	0.9793	128	2	128	relu	0.066	0.00191
18	0.9735	32	3	128	tanh	0.209	0.00034
19	0.9420	64	2	32	relu	0.171	0.00011

O melhor trial foi o de número 7, com acurácia de validação de 0.9808, usando `batch_size=128`, `n_layers=2`, `units=128`, `activation=relu`, `drop_rate=0.1125` e `learning_rate=0.002429`.

3. Modelo final e avaliação no conjunto de teste

Reconstruí o modelo final com os hiperparâmetros do trial 7 e treinei por 25 épocas, dessa vez sem a restrição de tempo dos 20 trials. A arquitetura ficou: Flatten(784) → Dense(128, relu) → Dropout(0.1125) → Dense(128, relu) → Dropout(0.1125) → Dense(10, softmax), totalizando 118.282 parâmetros treináveis (cerca de 462 KB).

No conjunto de teste, com as 10.000 imagens que o modelo nunca tinha visto, a acurácia ficou em 0.9788 e a perda em 0.1058.

classification_report (teste):

Classe	Precisão	Recall	F1-score	Suporte
0	0.99	0.99	0.99	980
1	0.99	0.99	0.99	1135
2	0.98	0.98	0.98	1032
3	0.97	0.98	0.98	1010
4	0.99	0.97	0.98	982
5	0.96	0.98	0.97	892
6	0.98	0.98	0.98	958
7	0.97	0.98	0.98	1028
8	0.99	0.96	0.97	974
9	0.96	0.98	0.97	1009
acurácia geral			0.98	10000

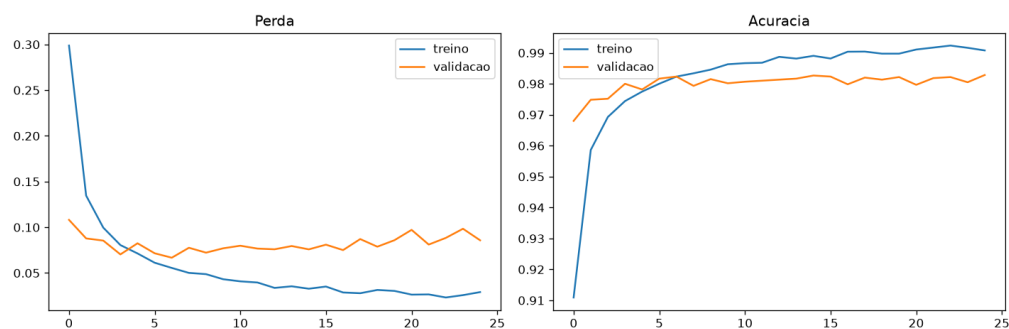


Figura 1 — Curvas de perda e acurácia (treino x validação) ao longo das 25 épocas.

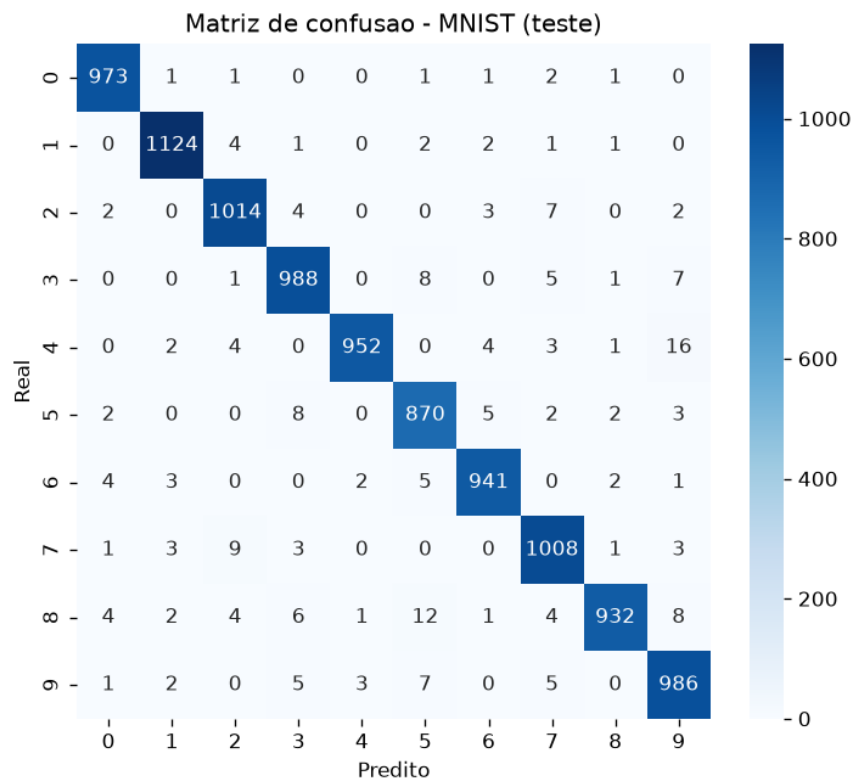


Figura 2 — Matriz de confusão do modelo final no conjunto de teste.

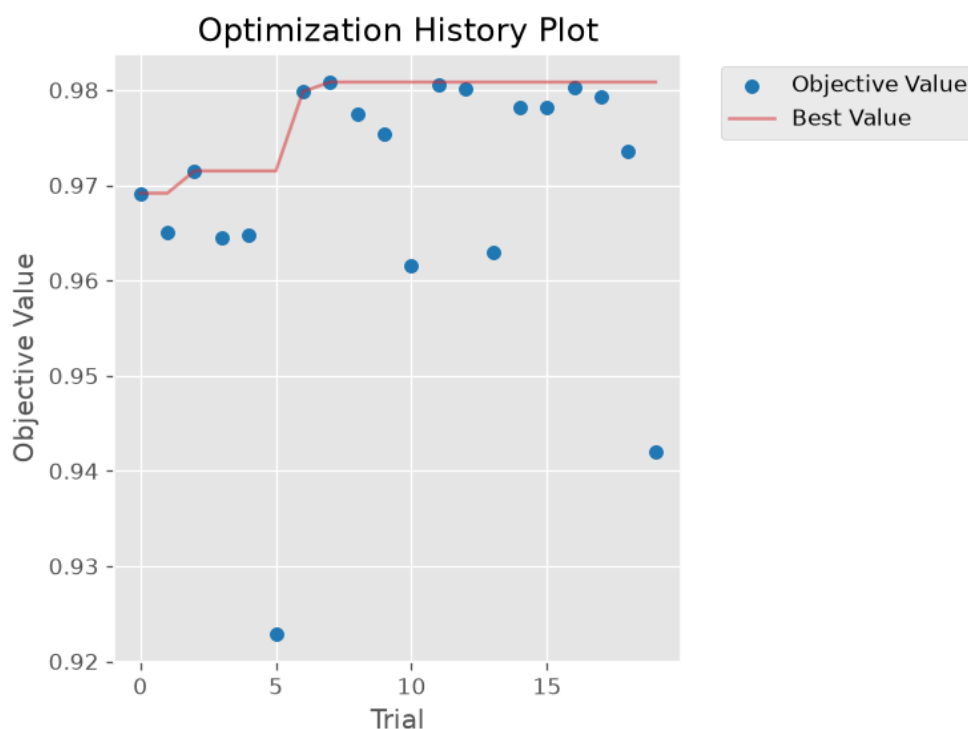


Figura 3 — Histórico de otimização do Optuna (valor objetivo por trial).

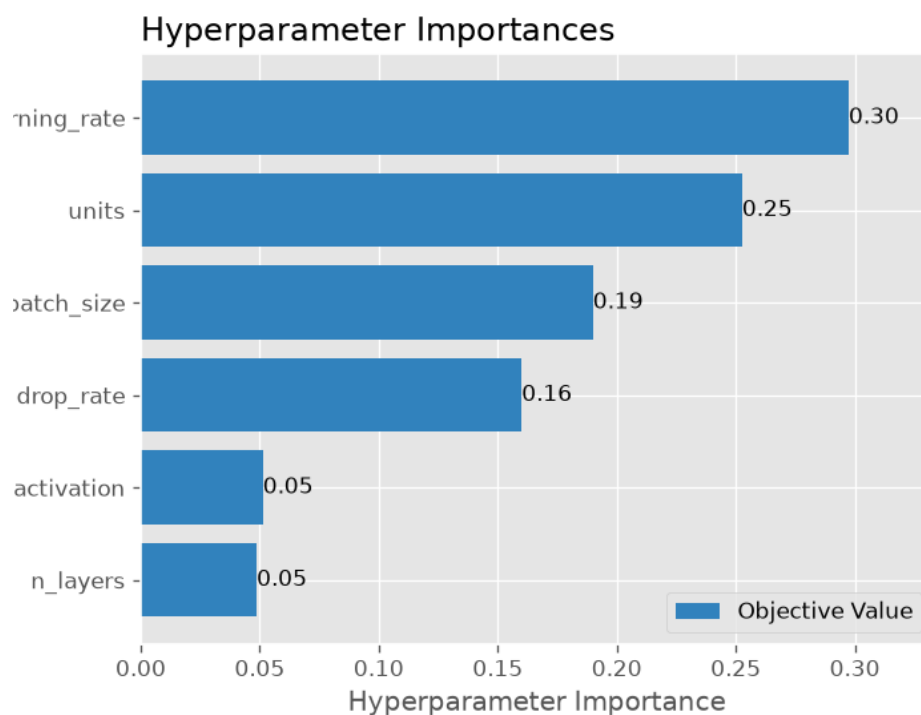


Figura 4 — Importância de cada hiperparâmetro na busca do Optuna.

4. Comentário sobre o ajuste final e sua qualidade

Dando uma olhada nos 20 trials, um padrão aparece bem rápido: os melhores resultados, com acurácia de validação acima de 0.978, quase sempre têm `n_layers` igual a 1 ou 2, `units`=128 e `activation`=relu, com `learning_rate` numa faixa entre 5e-4 e 2.5e-3, e `batch_size` de 32 ou 128.

Mais unidades por camada (128) já dão conta do MNIST sem precisar de muitas camadas — o `n_layers=3` não trouxe ganho nenhum de forma consistente, os trials 2, 5, 15 e 18 variam bastante entre si, o que sugere que essa profundidade extra não compensa em só 8 épocas de treino. A ReLU também superou a tanh quase sempre entre os melhores trials: a tanh (presente nos trials 1, 3, 8, 10 e 18) satura mais fácil e converge mais devagar nesse regime curto de treinamento. Já as taxas de aprendizado muito baixas, próximas de $1e-4$ (trials 10, 15 e 19), e a mais alta usada, de quase $1e-2$ (trial 13), tiveram os piores resultados: a primeira por não dar tempo de convergir, a segunda por deixar o treino instável. A faixa que funcionou bem ficou entre $5e-4$ e $2.5e-3$. Quanto ao dropout, o valor ótimo encontrado (0.1125) é relativamente baixo, o que indica que uma regularização mais leve já resolve bem no caso do MNIST.

Um ponto que achei interessante: os trials 7, 11, 12 e 16, todos com `units=128`, `n_layers=2`, `relu` e `learning rate` entre 0.001 e 0.0024, tiveram desempenho muito parecido entre si (entre 0.980 e 0.981). Isso me dá mais confiança de que a busca encontrou uma região estável de bons hiperparâmetros, e não só um pico isolado de sorte. O ganho em relação ao pior resultado da busca, o trial 5 com 0.9228, foi de mais de 5 pontos percentuais, e o resultado final no teste (0.9788) veio bem próximo do que a validação já indicava, o que mostra que o ajuste generalizou bem e não ficou preso aos dados usados durante a busca.

Tarefa 3 — CNNs (PyTorch) para o Multiprova Corretor (EMNIST)

Notebook: `tarefa3_cnn_emnist_pytorch.ipynb`, executado localmente em Python 3.13 / Windows, com PyTorch 2.14.0, em CPU. Reproduzi em PyTorch a melhor arquitetura de CNN do artigo de Silva Filho et al. (2022) sobre o app Multiprova Corretor, aplicando ela a 3 subproblemas montados a partir do EMNIST.

1. Arquitetura

A rede (chamei de `cnnMultiprova`) tem 4 blocos de `Conv2d(3x3)` seguido de `ReLU` e `MaxPool2d(2)`, com 32, 64, 64 e 64 filtros respectivamente, e termina com `Flatten` e uma camada `Linear(256, n_classes)`. As imagens do EMNIST entram redimensionadas para 32×32, em 1 canal. Para o treinamento usei Adam ($\text{lr}=1\text{e-}3$), `CrossEntropyLoss`, batch size 128 e até 12 épocas, com early stopping de paciência 3 olhando a perda de validação.

2. Subproblemas e resultados

No subproblema dos dígitos de 1 a 5, usei o split "digits" do EMNIST (102.000 imagens de treino, 18.000 de validação e 20.000 de teste depois de filtrar e separar). O treino parou em 7 épocas por early stopping e a acurácia de teste ficou em 0.9975, com precisão, recall e f1 de 1.00 em todas as 5 classes no `classification_report`.

Para o problema "V ou F" usei o split "letters", filtrando só as letras v e f (8.160 treino, 1.440 validação, 1.600 teste). Convergiu em 11 épocas e a acurácia de teste bateu 1.0000, ou seja, 100% — um problema binário mais simples, com bastante exemplo de cada classe, então esse resultado não chega a surpreender.

Já no subproblema das letras A a E, também do split "letters" (20.400 treino, 3.600 validação, 4.000 teste), o treino levou 9 épocas até parar e a acurácia de teste foi 0.9782. A letra A foi a que teve o pior recall (0.95), provavelmente por se confundir visualmente com outras letras manuscritas parecidas.

Subproblema	Épocas até parar	Acc. teste
Dígitos (1-5)	7	0.9975
V ou F	11	1.0000
Letras (A-E)	9	0.9782

3.1) Tabela: parâmetros treináveis e tamanho dos modelos exportados (.pt)

Modelo	Parâmetros treináveis	Tamanho (MB)
Dígitos (1-5)	93.957	0.363
V ou F	93.186	0.359
Letras (A-E)	93.957	0.363

Os três modelos ficaram com praticamente o mesmo tamanho — a diferença de cerca de 770 parâmetros entre os modelos de 5 classes e o de 2 classes vem só da última camada densa, a `Linear(256, n_classes)`. Isso mostra que quem domina o custo do modelo são as camadas convolucionais, que são iguais nos três casos, e não a camada de saída. No fim, os modelos ficaram bem enxutos, todos abaixo de 0.4 MB, o que combina bem com um uso embarcado em um app de celular como o Multiprova

Corretor.

3.2) Como o tamanho do modelo poderia ser reduzido?

Existem várias formas de atacar isso, da mais simples até a mais trabalhosa. A mais óbvia é reduzir a própria arquitetura, usando menos filtros por camada convolucional (por exemplo 16/32/32/32 em vez de 32/64/64/64) ou até removendo alguma camada. Outra opção bem eficiente é a quantização: converter os pesos de float32 para int8 ou float16 costuma reduzir o tamanho em até 4 vezes com perda quase nenhuma de acurácia. Também dá pra aplicar poda (pruning), removendo pesos ou filtros cuja magnitude já é próxima de zero, o que deixa a rede mais esparsa.

Em arquiteturas maiores, trocar as convoluções normais por convoluções separáveis em profundidade (como faz o MobileNet) reduz bastante o número de parâmetros por camada. Outra alternativa é a destilação de conhecimento, em que uma rede menor é treinada para imitar as saídas de uma rede maior já treinada. Trocar o Flatten seguido de uma camada densa grande por um Global Average Pooling também ajuda a cortar parâmetros, embora nesse caso específico a camada final já seja pequena. Fatoração de baixo posto nas matrizes de peso e exportação para formatos mais compactos como ONNX ou TensorFlow Lite completam a lista de opções, e ainda trazem otimizações de grafo que reduzem o tamanho final do arquivo.

Como os modelos que treinei aqui já são pequenos, abaixo de 0.4 MB cada, acho que a estratégia com melhor custo-benefício para um app como o Multiprova Corretor seria aplicar quantização pós-treinamento para INT8, que reduz o tamanho em cerca de 4 vezes sem precisar retreinar nada, combinada com a exportação para um formato mais compacto como ONNX Runtime Mobile ou TFLite. Poda, destilação ou uma arquitetura ainda menor eu deixaria como próximo passo, só se surgisse alguma restrição de memória mais severa.

Links dos Notebooks e Repositório

Repositório GitHub (código, notebooks executados e relatórios):

<https://github.com/lucassmsantoss/aprendizado-profundo>

Notebook Tarefa 2 (Google Colab):

https://colab.research.google.com/drive/1zFKRRcnSVH_FaU9bMy0xza6bvzjJmGZG?usp=sharing

Notebook Tarefa 3 (Google Colab):

https://colab.research.google.com/drive/1j0Qyu3_pJlpritZf15XOShYtwKJQGQu_?usp=sharing